INT1339 - Ngôn ngữ Lập trình C++

OOP
Sự Thừa Kế

Giảng viên: Lê Hà Thanh

lehathanh@ptithcm.edu.vn

Nội dung

- Lập trình hướng đối tượng
- Các lớp cơ sở và lớp dẫn suất
- Các hàm ảo
- Kiểm soát truy xuất lớp và thừa kế
- Các kiểu thừa kế
- Đa thừa kế

Lớp Dẫn xuất và lớp Cơ sở

Viết chương trình quản lý thông tin nhân viên một công ty

- Cấu trúc dữ liệu nhân viên

```
struct Employee {
        string first_name, family_name;
        char middle_initial;
        Date hiring_date;
        short department;
        // ...
};
```

- Định nghĩa người quản lý:

```
struct Manager {
        Employee emp;                    // manager's employee record
        list<Employee*> group;   // people managed
        short level;
        // ...
};
```

Lớp Dẫn xuất và lớp Cơ sở

Cần một khai báo rõ ràng rằng người quản lý **Manager** cũng là một nhân viên **Employee**

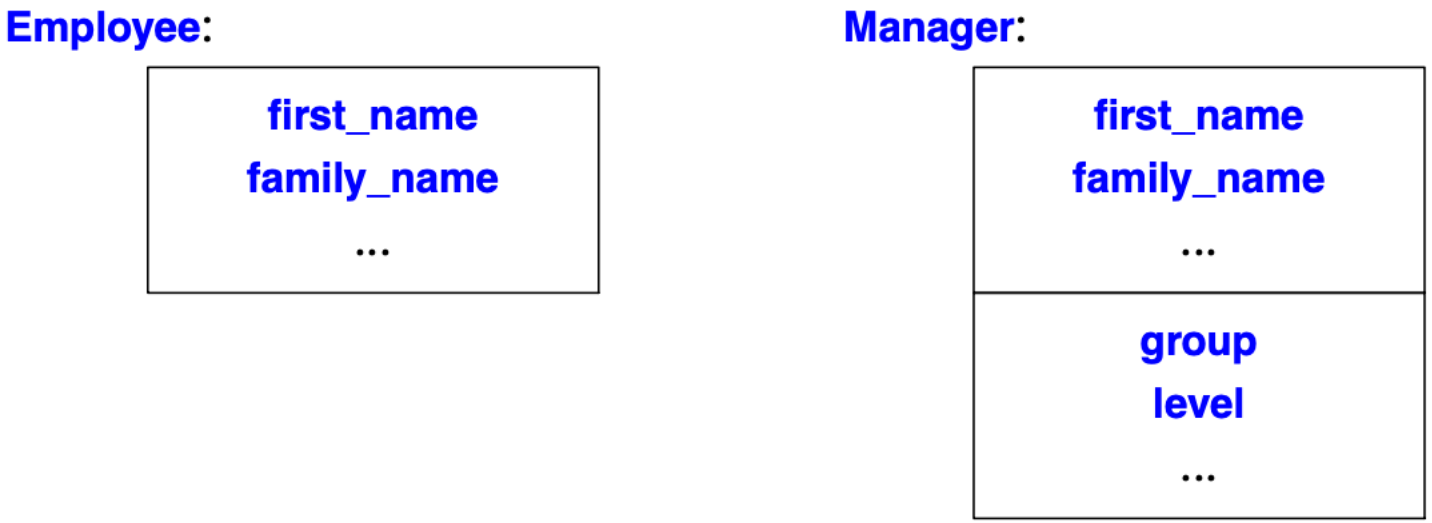
```
struct Manager : public Employee {
        list<Employee*> group;
        short level;
        // ...
};
```

- **Manager** được dẫn xuất (*derived*) từ **Employee**.
  Hay phát biểu ngược lại, **Employee** là lớp cơ sở (*base class*) cho **Manager**.

- Thành viên của lớp dẫn xuất: Lớp **Manager** có
  - Các thành viên của lớp **Employee** (**first_name**, **department**, v.v.) và
  - Các thành viên riêng của nó (**group**, **level**, v.v.)

Sự Kế thừa (Inheritance)

- Lớp dẫn xuất kế thừa các thuộc tính từ lớp cơ sở của nó.
- Lớp cơ sở : siêu lớp (*superclass*)
- Lớp dẫn xuất : lớp con (*subclass*)
- Lớp dẫn xuất thường lớn hơn lớp cơ sở của nó theo nghĩa nó chứa nhiều dữ liệu hơn và cung cấp nhiều chức năng hơn



Sử dụng

```
void f(Manager m1, Employee e1)
{
        list<Employee*> elist {&m1,&e1);
        // ...
}
```

Vì một Manager cũng là một Employee ,

→ một con trỏ Manager* có thể được sử dụng như con trỏ Employee*

→ Manager& có thể được sử dụng như Employee&

Điều ngược lại, một Employee không nhất thiết là một Manager

→ Employee* không thể được sử dụng như Manager* .

Tổng quát:

- Nếu một lớp dẫn xuất **Derived** có lớp cơ sở công khai là **Base**,

  thì **Derived*** có thể được gán cho một biến kiểu **Base*** mà không cần dùng tới phép chuyển kiểu tường minh.

- Phép chuyển **Base*** thành **Derived*** phải là tường minh

Ví dụ

```
void g(Manager mm, Employee ee)
{
        Employee* pe = &mm;                 // OK: every Manager is an Employee
        Manager* pm = &ee;                  // error: not every Employee is a Manager

        pm->level = 2;                      // disaster: ee doesn't have a level

        pm = static_cast<Manager*>(pe);     // brute force: works because pe points
                                            // to the Manager mm

        pm->level = 2;                      // fine: pm points to the Manager mm that has a level
}
```

Chú ý: Một đối tượng của lớp dẫn xuất có thể được xem là đối tượng của lớp cơ sở thông qua con trỏ và tham chiếu.

Ngược lại là không đúng!

Các hàm thành viên trong kế thừa

```
class Employee {
public:
        void print() const;
        string full_name() const { return first_name + ' ' + middle_initial + ' ' + family_name; }
        // ...
private:
        string first_name, family_name;
        char middle_initial;
        // ...
};

class Manager : public Employee {
public:
        void print() const;
        // ...
};
```

Các hàm thành viên trong kế thừa

- Một thành viên của lớp dẫn xuất có thể sử dụng các thành viên *public* và *protected* của lớp cơ sở nếu các thành viên này được khai báo trong lớp dẫn xuất.

```
void Manager::print() const
{
        cout << "name is " << full_name() << '\n';
        // ...
}
```

- Lớp dẫn xuất <u>không truy xuất</u> được các thành viên *private* của lớp cơ sở.

```
void Manager::print() const
{
        cout << " name is " << family_name << '\n';          // error!
        // ...
}
```

→ Có thể gọi các thành viên public của lớp cơ sở

```
void Manager::print() const
{
        Employee::print();   // print Employee information
        cout << level;       // print Manager-specific information
        // ...
}
```

Hàm khởi tạo và Hàm hủy trong kế thừa

- Hàm khởi tạo và hàm hủy của lớp cơ sở không được kế thừa.

- Các đối tượng được tạo từ dưới lên (bottom up): đối tượng cơ sở -> thành viên -> đối tượng lớp dẫn xuất.
  Các đối tượng được hủy từ trên xuống (top down): đối tượng lớp dẫn xuất -> thành viên -> đối tượng cơ sở;
- Mỗi lớp có thể khởi tạo các thành viên của mình (nhưng không khởi tạo trực tiếp các thành viên hoặc lớp cơ sở của lớp cơ sở của nó);
- Các destructors trong phân cấp kế thừa cần được khai báo **virtual**;
- Copy constructors của các lớp trong phân cấp kế thừa nên được dùng cẩn thận để tránh gây cắt lát;
- Lời gọi hàm ảo với **dynamic_cast** hoặc **typeid()** trong constructor/destructor phản ánh một giai đoạn của việc tạo và hủy đối tượng

Kiến trúc phân cấp lớp

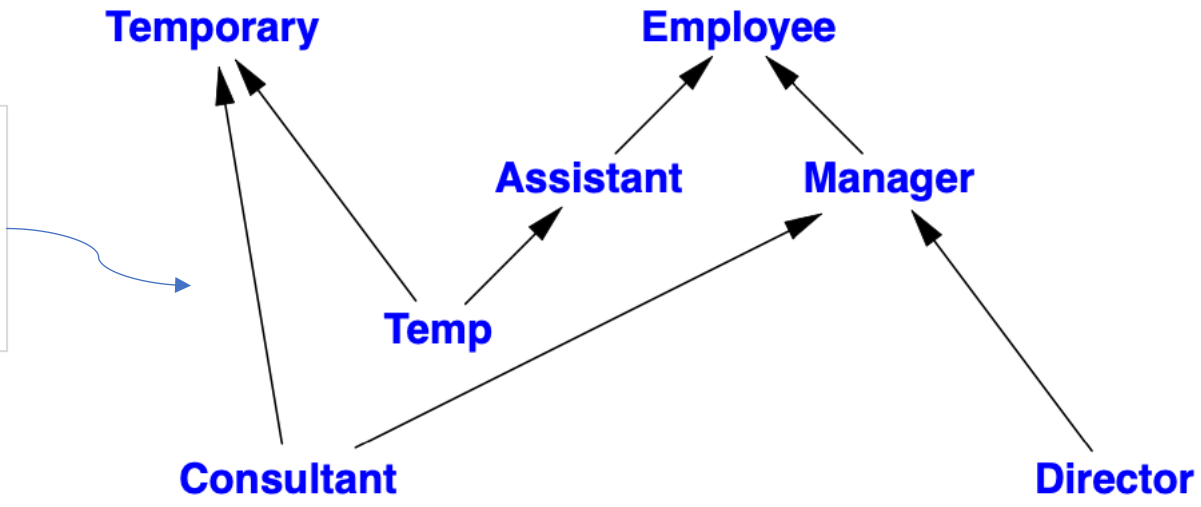
Phân cấp lớp

- Một lớp dẫn xuất có thể là một lớp cơ sở.

```
class Employee { /* ... */ };
class Manager : public Employee { /* ... */ };
class Director : public Manager { /* ... */ };
```

- Các lớp kế thừa nhau tạo thành kiến trúc phân cấp lớp

```
class Temporary { /* ... */ };
class Assistant : public Employee { /* ... */ };
class Temp : public Temporary, public Assistant { /* ... */ };
class Consultant : public Temporary, public Manager { /* ... */ };
```



Hàm ảo

Cho phép khai báo hàm trong lớp cơ sở và sau đó có thể định nghĩa lại hàm (ghi đè – override) trong lớp dẫn xuất.

- Trình biên dịch và trình liên kết sẽ đảm bảo sự tương ứng chính xác giữa đối tượng và các hàm được áp dụng cho đối tượng.

Hàm ảo – Ví dụ

virtual void print() const :

- print() đóng vai trò là giao diện cho hàm print() được định nghĩa trong lớp Employee và các hàm print() được định nghĩa trong các lớp dẫn xuất từ Employee.

```
class Employee {
public:
        Employee(const string& name, int dept);
        virtual void print() const;
        // ...
private:
        string first_name, family_name;
        short department;
        // ...
};
```

Hàm ảo – Quy tắc Khai báo

- Kiểu của đối số trong khai báo hàm ảo ở lớp dẫn xuất phải tương đồng với khai báo hàm trong lớp cơ sở.

- Hàm ảo phải được định nghĩa ở lớp cơ sở nơi nó được khai báo đầu tiên.
  Ví dụ:

```
void Employee::print() const
{
        cout << family_name << '\t' << department << '\n';
        // ...
}
```

Hàm ảo – Quy tắc Khai báo

- Hàm ảo có thể được sử dụng không phụ thuộc vào lớp có các lớp dẫn xuất hay không.
- Nếu lớp dẫn xuất không cần dùng hàm ảo thì có thể bỏ qua không cần khai báo hàm này. Chỉ cung cấp hàm nếu cần thiết.

```
class Manager : public Employee {
public:
        Manager(const string& name, int dept, int lvl);
        void print() const;
        // ...
private:
        list<Employee*> group;
        short level;
        // ...
};

void Manager::print() const
{
        Employee::print();
        cout << "\tlevel " << level << '\n';
        // ...
}
```

Hàm ghi đè (override functions)

Một hàm trong lớp dẫn xuất có cùng tên và cùng tập các kiểu đối số (chữ ký của hàm) với hàm ảo trong lớp cơ sở được gọi là ghi đè lên phiên bản ở lớp cơ sở của hàm ảo.

- Hàm ghi ghi đè được phép trả về một kiểu là biến thể của kiểu trả về nguyên thủy.

- Lời gọi hàm ảo có thể thực hiện tường minh tới hàm ảo với tham chiếu từ lớp sở hữu (ví dụ Employee::print()), hoặc gọi tới hàm ghi đè tương ứng với đối tượng đang hoạt động.

Hàm ghi đè – Ví dụ

```
void print_list(const list<Employee*>& s)
{
        for (auto x : s)
                x->print();
}
```

```
int main()
{
        Employee e {"Brown",1234};
        Manager m {"Smith",1234,2};

        print_list({&e,&m});
}
```

```
Smith 1234
        level 2
Brown 1234
```

Đa hình - Polymorphism

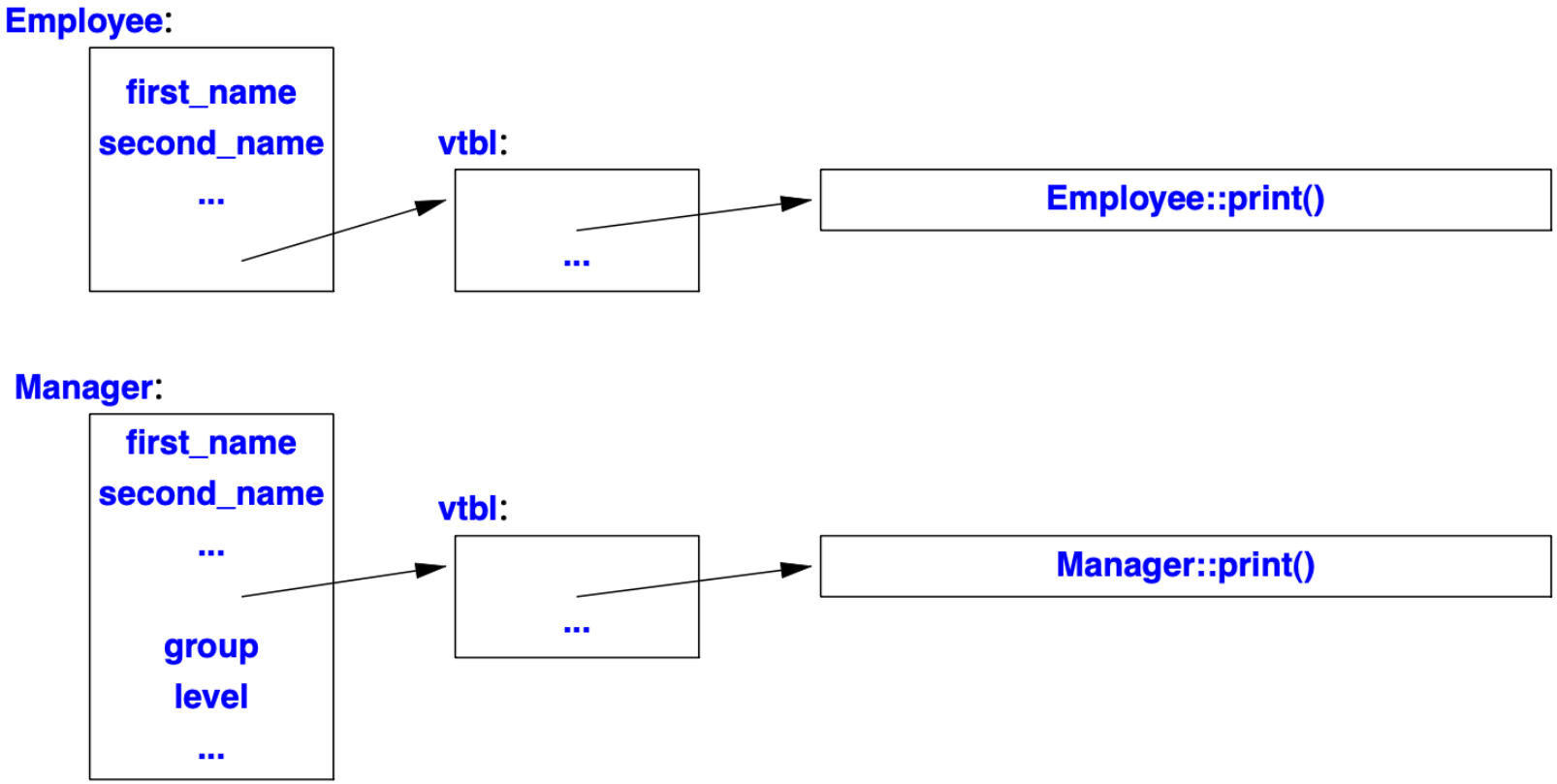
Là việc đối tượng (của một lớp) có các hành vi khác nhau trong các tình huống khác nhau.

- Xác định hành vi đúng của một lớp độc lập với kiểu lớp nào được sử dụng.
- Khai báo hàm ảo được gọi là kiểu đa hình, hay còn gọi là kiểu đa hình khi thực thi chương trình (run-time polymorphic type).

Cách gọi hàm trong cơ chế đa hình

Trình biên dịch chuyển đổi tên của một hàm ảo thành một chỉ mục (index) trong một bảng các con trỏ trỏ tới các hàm. Bảng này gọi là bảng hàm ảo (vtbl).

Mỗi lớp có các hàm ảo đều có vtbl riêng để xác định các hàm ảo của nó.



Các loại đa hình

- Đa hình khi biên dịch (compile-time polymorphism)
  - Cơ chế chồng hàm (function overloading): các hàm cùng tên, khác tham số.
  - Trình biên dịch xác định lời gọi hàm cụ thể để liên kết với đối tượng sau khi suy ra đối tượng tại thời điểm chạy.

- Đa hình khi thực thi (runtime polymorphism)
  - Cơ chế ghi đè hàm (function overriding): ở lớp dẫn xuất có định nghĩa lại một hoặc nhiều hàm ở lớp cơ sở.
  - Chỉ xác định được lời gọi hàm tại thời điểm thực thi.
  - Còn gọi là kết nối muộn (late binding) hay đa hình động

Kiểm soát cơ chế ghi đè

Sử dụng các chỉ định sau:

- virtual: cho biết hàm có thể được ghi đè.
- =0: Hàm phải được khai báo virtual và phải được ghi đè.
- override: Hàm phải ghi đè một hàm ảo trong lớp cơ sở.
- final: Hàm không được phép ghi đè.

Nếu không dùng các chỉ định trên, một hàm thành viên non-static sẽ là hàm ảo nếu và chỉ nếu nó ghi đè một hàm được chỉ định virtual trong lớp cơ sở.

Chỉ định override

Chỉ định override không phải là một phần của kiểu hàm và không được lặp lại trong định nghĩa hàm nằm bên ngoài lớp.

```
class Derived : public Base {
        void f() override;          // OK if Base has a virtual f()
        void g() override;          // OK if Base has a virtual g()
};

void Derived::f() override          // error: override out of class
{
        // ...
}

void g()                            // OK
{
        // ...
}
```

Chỉ định final

Để dừng ý định cho phép ghi đè lên hàm virtual.

```
struct Node {    // interface class
        virtual Type type() = 0;
        // ...
};

class If_statement : public Node {
public:
        Type type() override final;        // prevent further overriding
        // ...
};
```

```
class Modified_if_statement : public If_statement  {
public:
        Type type() override;        // error: If_statement::type() is final
        // ...
};
```

Chỉ định final

Để dừng ý định cho phép ghi đè lên hàm virtual. Áp dụng cho toàn bộ các hàm trong lớp.

```
class For_statement final : public Node {
public:
        Type type() override;
        // ...
};

class Modified_for_statement : public For_statement  {        // error: For_statement is final
        Type type() override;
        // ...
};
```

Chỉ định final

Chỉ định final không phải là một phần của kiểu hàm và không được lặp lại trong định nghĩa hàm nằm bên ngoài lớp.

```
class Derived : public Base {
        void f() final;          // OK if Base has a virtual f()
        void g() final;          // OK if Base has a virtual g()
        // ...
};

void Derived::f() final          // error: final out of class
{
        // ...
}

void g() final                   // OK
{
        // ...
}
```

Bổ sung các thành viên lớp cơ sở bằng using

Các hàm không ghi chồng vượt phạm vi của lớp.

Ví dụ:

```
struct Base {
        void f(int);
};

struct Derived : Base {
        void f(double);
};

void use(Derived d)
{
        d.f(1);             // call Derived::f(double)
        Base& br = d
        br.f(1);            // call Base::f(int)
}
```

Bổ sung các thành viên lớp cơ sở bằng using

Có thể bổ sung hàm vào lớp bằng khai báo using .

→ Bổ sung vào không gian tên (namespace) của lớp.

Ví dụ:

```
struct Base {
        void f(int);
};

struct D2 : Base {
        using Base::f;          // bring all fs from Base into D2
        void f(double);
};

void use2(D2 d)
{
        d.f(1);                 // call D2::f(int), that is, Base::f(int)
        Base& br = d
        br.f(1);                // call Base::f(int)
}
```

Bổ sung các thành viên lớp cơ sở bằng using

Có thể bổ sung các hàm từ nhiều lớp cơ sở.

Ví dụ:

```
struct B1 {
        void f(int);
};

struct B2 {
        void f(double);
};

struct D : B1, B2 {
        using B1::f;
        using B2::f;
        void f(char);
};

void use(D d)
{
        d.f(1);       // call D::f(int), that is, B1::f(int)
        d.f('a');     // call D::f(char)
        d.f(1.0);     // call D::f(double), that is, B2::f(double)
}
```

Kế thừa các hàm tạo

Ví dụ: tạo một lớp vector mới tương tự std::vector nhưng có thêm phép kiểm tra miền giá trị kích thước vector.

```
template<class T>
struct Vector : std::vector<T> {
        T& operator[](size_type i) { check(i); return this->elem(i); }
        const T& operator[](size_type i) const { check(i); return this->elem(i); }

        void check(size_type i) { if (this->size()<i) throw range_error{"Vector::check() failed"}; }
};
```

Khai báo này đã không kế thừa các hàm tạo từ std::vector, nên gây lỗi khởi tạo vector.

```
Vector<int> v { 1, 2, 3, 5, 8 };    // error: no initializer-list constructor
```

Kế thừa các hàm tạo

```
template<class T>
struct Vector : std::vector<T> {
    using vector<T>::vector;        // inherit constructors

    T& operator=[](size_type i) { check(i); return this->elem(i); }
    const T& operator=(size_type i) const { check(i); return this->elem(i); }

    void check(size_type i) { if (this->size()<i) throw Bad_index(i); }
};
```

```
Vector<int> v { 1, 2, 3, 5, 8 };        // OK: use initializer-list constructor from std::vector
```

Kế thừa các hàm tạo - Cẩn thận khi khởi tạo biến thành viên trong lớp kế thừa

```
struct B1 {
        B1(int) { }
};

struct D1 : B1 {
        using B1::B1;   // implicitly declares D1(int)
        string s;       // string has a default constructor
        int x;          // we "forgot" to provide for initialization of x
};

void test()
{
        D1 d {6};       // oops: d.x is not initialized
        D1 e;           // error: D1 has no default constructor
}
```

Kế thừa các hàm tạo - Cẩn thận khi khởi tạo biến thành viên trong lớp kế thừa

Cách khắc phục

```
struct B1 {
        B1(int) { }
};
```

```
struct D1 : B1 {
        D1(int i) : B1(i) { }
        string s;          // string has a default constructor
        int x;             // we "forgot" to provide for initialization of x
};
```

```
struct B1 {
        B1(int) { }
};
```

```
struct D1 : B1 {
        using B1::B1;  // implicitly declares D1(int)
        int x {0};     // note: x is initialized
};

void test()
{
        D1 d {6};  // d.x is zero
}
```

Các Lớp Trừu tượng
Abstract Classes

Các khái niệm và Định nghĩa

- Một hàm ảo được định nghĩa “thuần ảo” bằng chỉ định =0.

Một lớp có một hoặc nhiều hàm ảo thuần được gọi là **lớp trừu tượng**.

- Lớp trừu tượng là giao diện cho các đối tượng được truy cập thông qua con trỏ và tham chiếu (nhằm bảo toàn tính chất đa hình).
  → Lớp trừu tượng có một hàm hủy ảo.
- Không có đối tượng nào được tạo ra từ lớp trừu tượng
  → Lớp trừu tượng không có hàm tạo

Ví dụ AC_01

```
class Shape {          // abstract class
public:
        virtual void rotate(int) = 0;              // pure virtual function
        virtual void draw() const = 0;             // pure virtual function
        virtual bool is_closed() const = 0;        // pure virtual function
        // ...
        virtual ~Shape();                          // virtual
};
```

Ví dụ AC_02

- Lớp trừu tượng chỉ có thể được sử dụng như là một giao diện cho các lớp khác.

```
class Point { /* ... */ };

class Circle : public Shape {
public:
        void rotate(int) override { }
        void draw() const override;
        bool is_closed() const override { return true; }

        Circle(Point p, int r);
private:
        Point center;
        int radius;
};
```

Lớp dẫn xuất trừu tượng

- Một hàm thuần ảo không được định nghĩa lại ở lớp kế thừa (là lớp dẫn xuất) vẫn giữ nguyên tính chất hàm thuần ảo

→ Lớp dẫn xuất cũng là lớp trừu tượng.

Ví dụ:

```
class Polygon : public Shape {              // abstract class
public:
        bool is_closed() const override { return true; }
        // ... draw and rotate not overridden ...
};

Polygon b {p1,p2,p3,p4};        // error: declaration of object of abstract class Polygon
```

Khai báo khắc phục lỗi cho Polygon

```
class Irregular_polygon : public Polygon {
        list<Point> lp;
public:
        Irregular_polygon(initializer_list<Point>);

        void draw() const override;
        void rotate(int) override;
        // ...
};

Irregular_polygon poly {p1,p2,p3,p4};     // assume that p1 .. p4 are Points defined somewhere
```

Lớp trừu tượng làm giao diện

- Lớp trừu tượng cung cấp một giao diện.
- Chi tiết triển khai được xác lập trong các lớp kế thừa

Ví dụ: Hệ điều hành ẩn các chi tiết của các trình điều khiển thiết bị (device drivers) sau lớp trừu tượng.

Sau đó mới chỉ định các trình điều khiển là lớp dẫn xuất từ giao diện, và thao tác điều khiển các trình khác nhau thông qua giao diện này.

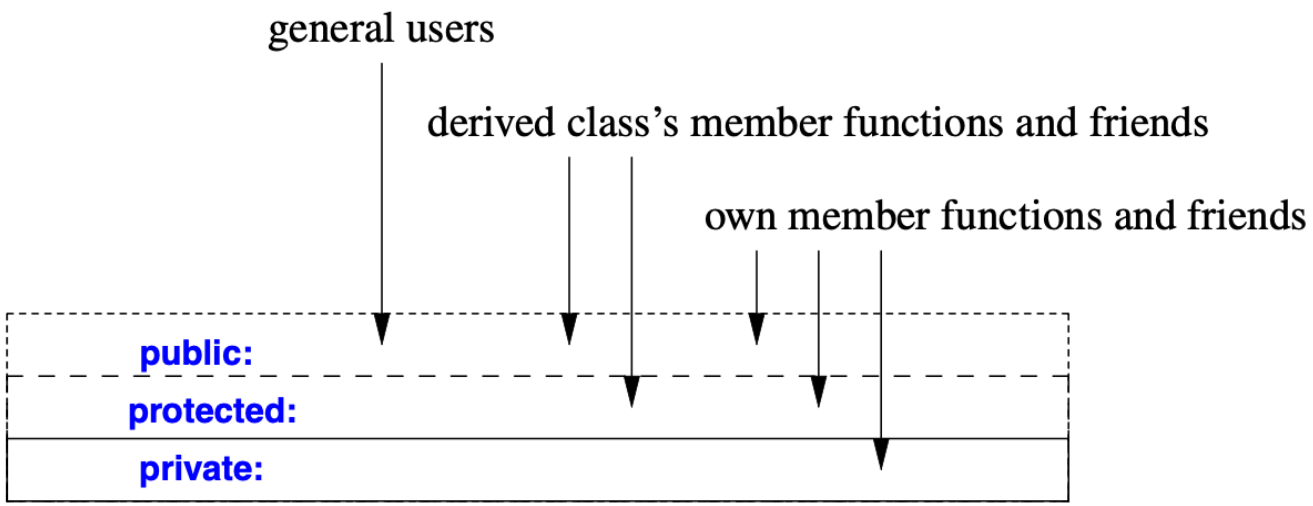
```
class Character_device {
public:
        virtual int open(int opt) = 0;
        virtual int close(int opt) = 0;
        virtual int read(char* p, int n) = 0;
        virtual int write(const char* p, int n) = 0;
        virtual int ioctl(int ...) = 0;              //device I/O control

        virtual ~Character_device() { }              // virtual destructor
};
```

Kiểm soát Truy cập vào thành viên của lớp

Phân loại thành viên theo khả năng truy cập

Các thành viên có thể được khai báo private, protected, hoặc public.

- private: chỉ được truy cập và sử dụng bởi các hàm thành viên và các hàm bạn của lớp chứa.
- protected: có thể được truy cập và sử dụng bởi các hàm thành viên và các hàm bạn của lớp chứa, và các hàm thành viên và bạn của các lớp dẫn xuất từ lớp này.
- public: được truy cập và sử dụng tự do.



Ví dụ - AccCtl_01

```
template<class T>
class List {
public:
        void insert(T);
        T get();
        // ...
private:
        struct Link { T val; Link* next; };

        struct Chunk {
                enum { chunk_size = 15 };
                Link v[chunk_size];
                Chunk* next;
        };

        Chunk* allocated;
        Link* free;
        Link* get_free();
        Link* head;
};
```

Ví dụ - AccCtl_01

```
template<class T>
void List<T>::insert(T val)
{
        Link* lnk = get_free();
        lnk->val = val;
        lnk->next = head;
        head = lnk;
}

template<class T>
T List<T>::get()
{
        if (head == 0)
                throw Underflow{};  // Underflow is my exception class

        Link* p= head;
        head = p->next;
        p->next = free;
        free = p;
        return p->val;
}
```

Ví dụ - AccCtl_01

```
template<class T>
typename List<T>::Link* List<T>::get_free()
{
        if (free == 0) {
                // ... allocate a new chunk and place its Links on the free list ...
        }
        Link* p = free;
        free = free->next;
        return p;
}
```

Ví dụ - AccCtl_01 – kiểm tra truy cập

- Các hàm không phải thành viên (ngoại trừ hàm bạn) đều không có quyền truy cập :

```
template<typename T>
void would_be_meddler(List<T>* p)
{
        List<T>::Link* q = 0;                          // error: List<T>::Link is private
        // ...
        q = p->free;                                   // error: List<T>::free is private
        // ...
        if (List<T>::Chunk::chunk_size > 31) {         // error: List<T>::Chunk::chunk_size is private
                // ...
        }
}
```

Truy cập đến các lớp cơ sở

Các lớp cơ sở được khai báo private, protected, hoặc public.

```
class X : public B { /* ... */ };
class Y : protected B { /* ... */ };
class Z : private B { /* ... */ };
```

Truy cập đến các lớp cơ sở

Giả sử lớp D dẫn xuất (kế thừa) B.

Khai báo lớp cơ sở B là:

- private: các thành viên public và protected của B chỉ được sử dụng bởi các hàm thành viên và bạn của D.
  Chỉ các thành viên và bạn của D mới đổi được D* thành B*.
- protected: các thành viên public và protected của B chỉ được sử dụng bởi các hàm thành viên và bạn của D, và các hàm thành viên và bạn của các lớp kế thừa từ D.
  Chỉ các thành viên và bạn của D và thành viên và bạn của các lớp kế thừa từ D mới đổi D* thành B*.
- public: các thành viên public của B được sử dụng bởi hàm bất kỳ.
  Thành viên protected có thể được sử dụng bởi các thành viên và bạn của D và thành viên và bạn của các lớp kế thừa từ D.
  Bất kỳ hàm nào cũng có thể đổi D* thành B*.

Đa kế thừa

```
struct B {
        int m;
        static int sm;
        // ...
};

class D1 : public virtual B { /* ... */ } ;
class D2 : public virtual B { /* ... */ } ;
class D12 : public D1, private D2 { /* ... */ };

D12* pd = new D12;
B* pb = pd;             // OK: accessible through D1
int i1 = pd->m;         // OK: accessible through D1
```

Khai báo using và kiểm soát truy cập

```
class B {
private:
        int a;
protected:
        int b;
public:
        int c;
};

class D : public B {
public:
        using B::a;             // error: B::a is private
        using B::b;             // make B::b publicly available through D
};
```

Khai báo using và kiểm soát truy cập

```
class BB : private B {      // give access to B::b and B::c, but not B::a
public:
        using B::b;
        using B::c;
};
```

Con trỏ tới các thành viên của lớp

Các toán tử truy xuất **->*** và **.***

Khai báo con trỏ tới thành viên lớp

- Dùng các toán tử `->*` và `.*`
- Con trỏ tới thành viên không thể gán tới `void*` hay các kiểu con trỏ thông thường.
- Con trỏ rỗng có thể được gán cho con trỏ tới thành viên.
  Ý nghĩa: “không thành viên”

Con trỏ tới các thành viên hàm

Ví dụ:

1. Khai báo lớp giao diện tiêu chuẩn cho các hàm chức năng (xem ➔)
2. Xử lý cụ thể của các chức năng sẽ được định nghĩa bởi đối tượng của lớp dẫn xuất, kế thừa từ Std_interface.

```
class Std_interface {
public:
        virtual void start() = 0;
        virtual void suspend() = 0;
        virtual void resume() = 0;
        virtual void quit() = 0;
        virtual void full_size() = 0;
        virtual void small() = 0;

        virtual ~Std_interface() {}
};
```

Con trỏ tới các thành viên hàm

- Dùng toán tử "địa chỉ của" - & - trỏ tới tên thành viên cụ thể của lớp.
  Vd: &Std_interface::suspend.
- Khai báo biến kiểu con trỏ trỏ tới thành viên của lớp X:
  X::*

```
using Pstd_mem = void (Std_interface::*)();   // pointer-to-member type

void f(Std_interface* p)
{
        Pstd_mem s = &Std_interface::suspend;      // pointer to suspend()
        p->suspend();                              // direct call
        p->*s();                                   // call through pointer to member
}
```

Con trỏ tới các thành viên hàm

- Toán tử ->* và .* cho phép một con trỏ tới thành viên có thể được dùng kết hợp với một một đối tượng.

  Vd:

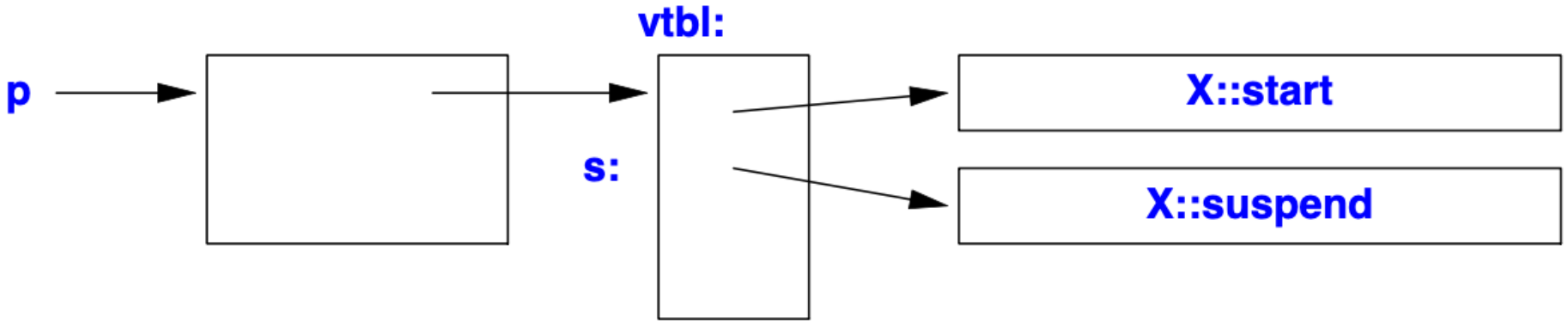
  p->*m liên kết **m** với đối tượng được trỏ bởi p.
  obj.*m liên kết **m** với đối tượng obj.

- Kết quả trả về của con trỏ tới thành viên hàm có kiểu tương ứng với kiểu trả về của hàm đã được định nghĩa.

- *Chú ý:* Không thể lưu lại kết quả của ->* hay .* để dùng sau đó!!

Con trỏ tới các thành viên hàm

Lời gọi **p->*s()** :



Con trỏ tới các thành viên hàm – Các lưu ý (1/2)

- Hàm được gọi qua con trỏ hàm có thể là hàm virtual.

Ví dụ: Nếu gọi tới hàm suspend() bằng một con trỏ hàm, sẽ được hàm suspend() đúng theo đối tượng mà con trỏ tới hàm đang áp dụng.

Ví dụ: Trong ví dụ sau dùng các con trỏ tới hàm thành viên để gọi các hàm được biểu diễn bằng các chuỗi "tên hàm" tương ứng.

```
map<string,Std_interface*> variable;
map<string,Pstd_mem> operation;

void call_member(string var, string oper)
{
    (variable[var]->*operation[oper])();// var.oper()
}
```

Con trỏ tới các thành viên hàm – Các lưu ý (2/2)

- Vì hàm thành viên khai báo static không có liên quan đến một đối tượng cụ thể nào, nên con trỏ tới một thành viên static chỉ được xem là một con trỏ thông thường.

Ví dụ:

```
class Task {
        // ...
        static void schedule();
};

void (*p)() = &Task::schedule;              // OK
void (Task::* pm)() = &Task::schedule;      // error: ordinary pointer assigned
                                            // to pointer to member
```

Con trỏ tới các thành viên dữ liệu

Cho khai báo một lớp cấu trúc C như sau:

```
struct C {
        const char* val;
        int i;

        void print(int x) { cout << val << x << '\n'; }
        int f1(int);
        void f2();
        C(const char* v) { val = v; }
};

using Pmfi = void (C::*)(int);    // pointer to member function of C taking an int
using Pm = const char* C::*;  // pointer to char* data member of C
```

Con trỏ tới các thành viên dữ liệu

Hãy phân tích đoạn chương trình:

```
void f(C& z1, C& z2)
{
        C* p = &z2;
        Pmfi pf = &C::print;
        Pm pm = &C::val;

        z1.print(1);
        (z1.*pf)(2);
        z1.*pm = "nv1 ";
        p->*pm = "nv2 ";
        z2.print(3);
        (p->*pf)(4);

        pf = &C::f1;      // error: return type mismatch
        pf = &C::f2;      // error: argument type mismatch
        pm = &C::i;       // error: type mismatch
        pm = pf;          // error: type mismatch
}
```

Các thành viên cơ sở và thành viên dẫn xuất

Trong một lớp dẫn xuất ít nhất sẽ có các thành viên nó kế thừa được từ các lớp cơ sở của mình.

- Và thường sẽ bổ sung thêm các thành viên riêng của nó.

→Khái niệm *contravariance*:

Có thể gán con trỏ tới một thành viên của một lớp cơ sở cho một con trỏ tới một thành viên của lớp dẫn xuất.

Điều ngược lại là không đúng!

```
class Text : public Std_interface {
public:
        void start();
        void suspend();
        // ...
        virtual void print();
private:
        vector s;
};

void (Std_interface::* pmi)() = &Text::print;     // error
void (Text::*pmt)() = &Std_interface::start;      // OK
```

Nguyên tắc đảm bảo an toàn khi dùng con trỏ

Nguyên tắc đảm bảo an toàn: một con trỏ không bao giờ có thể trỏ tới một đối tượng không có ít nhất các thuộc tính mà con trỏ đã hứa sẽ truy cập tới.

Ví dụ: xem ví dụ trang 6 và ví dụ trang 63.

Câu hỏi: hãy giải thích cách sử dụng con trỏ trong hai ví dụ trên.

Hết phần này!